COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 Data structure and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	7-28-2026	**DATE SUBMITTED:**	7-30-2026
NAME:	Mauro, Ryan Lewis B.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 4.1 **[Stack]**

1. PROCEDURE OUTPUT

```cpp
#include <iostream>
#include <stack> // calling stack from the STL

int main(){
    //create an object for the stack
    std::stack<int> newStack;

    std::cout<<"Stack Empty? "<<newStack.empty()<<std::endl;
    //push to the stack
    newStack.push(3);
    newStack.push(8);
    newStack.push(15);

    std::cout<<"Stack Empty? "<<newStack.empty()<<std::endl;
    std::cout<<"The size of the stack: "<<newStack.size()<<std::endl;
    std::cout<<"The top of the element is: "<<newStack.top()<<std::endl;
    std::cout<<"Removing elements from the stack: \n";
    newStack.pop();
    std::cout<<"The top of the element is: "<<newStack.top()<<std::endl;
    newStack.pop();
    std::cout<<"The top of the element is: "<<newStack.top()<<std::endl;
    newStack.pop();
    std::cout<<"Stack empty? "<<newStack.empty()<<std::endl;


    return 0;
}
```

Table 4-1. Code of ILO A

The program demonstrates how the Stack data structure works using the **Last In, Last Out** principle in C++. At the beginning, the stack is empty, which is confirmed by the output. Three values (3, 8, and 15) are then **pushed onto the stack**, making **15 the top element because it was added last.**

```
Stack Empty? 1
Stack Empty? 0
The size of the stack: 3
The top of the element is: 15
Removing elements from the stack:
The top of the element is: 8
The top of the element is: 3
Stack empty? 1
```

Table 4-2. Output of ILO A

The output confirms that the stack follows the **Last In, Last Out principle**. After inserting the values 3, 8, and 15, the top element is 15. As elements are removed, the top changes to 8, then 3, and finally the stack becomes empty again. This verifies that the last value added is always the first one removed.

```
    while (true) {
        std::cout << "\nStack Operations\n";
        std::cout << "1. PUSH\n";
        std::cout << "2. POP\n";
        std::cout << "3. TOP\n";
        std::cout << "4. isEMPTY\n";
        std::cout << "5. DISPLAY\n";
        std::cout << "Enter your choice: ";
        std::cin >> choice;

        switch (choice) {
            case 1: push(); break;
            case 2: pop(); break;
            case 3: Top(); break;
            case 4: std::cout << isEmpty() << std::endl; break;
            case 5: display(); break;    // Call display()
            default: std::cout << "Invalid choice." << std::endl;
        }
    }

    return 0;
}

bool isEmpty() {
    return (top == -1);
}

bool isFull() {
    return (top >= i - 1);
}

void push() {
    if (isFull()) {
        std::cout << "Stack Overflow" << std::endl;
        return;
    }

    std::cout << "Enter new value: ";
    std::cin >> newData;
    stack[++top] = newData;
}

void Top() {
    if (isEmpty()) {
        std::cout << "Stack Underflow." << std::endl;
        return;
    }

    std::cout << "The top element is: " << stack[top] << std::endl;
}

void pop() {
    if (isEmpty()) {
        std::cout << "Stack Underflow." << std::endl;
        return;
    }

    std::cout << "Popping: " << stack[top] << std::endl;
    top--;
}
```

Table 4-3. Functions for ILOB

The code demonstrates a menu-driven stack implemented using an array in C++. The program repeatedly displays a menu that allows the user to perform basic stack operations such as **push**, **pop**, view the **top** element, and check if the stack is empty. The **push()** function adds a new element to the top of the stack, while the **pop()** function removes the most recently added element. The **Top()** function shows the current top element without removing it, and the program uses **isFull()** and **isEmpty()** to prevent stack overflow and stack underflow, ensuring that all operations are performed safely and correctly.

```cpp
// New function to display all elements
void display() {
    if (isEmpty()) {
        std::cout << "Stack is empty." << std::endl;
        return;
    }

    std::cout << "Stack elements (top to bottom):" << std::endl;
    for (int j = top; j >= 0; j--) {
        std::cout << stack[j] << " ";
    }
    std::cout << std::endl;
}
```

Table 4-4. Modification for ILOB

The program implements a stack using an array and allows users to perform basic stack operations through a menu. Users can **push**, **pop**, view the **top** element, check if the stack is **empty**, and **display** all stack elements. It also **prevents stack overflow when the stack is full** and stack underflow when attempting to remove or view elements from an empty stack.

```
Enter the number of the max elements for new Stack: 10

Stack Operations
1. PUSH
2. POP
3. TOP
4. isEMPTY
5. DISPLAY
Enter your choice: 1
Enter new value: 67

Stack Operations
1. PUSH
2. POP
3. TOP
4. isEMPTY
5. DISPLAY
Enter your choice: 3
The top element is: 67

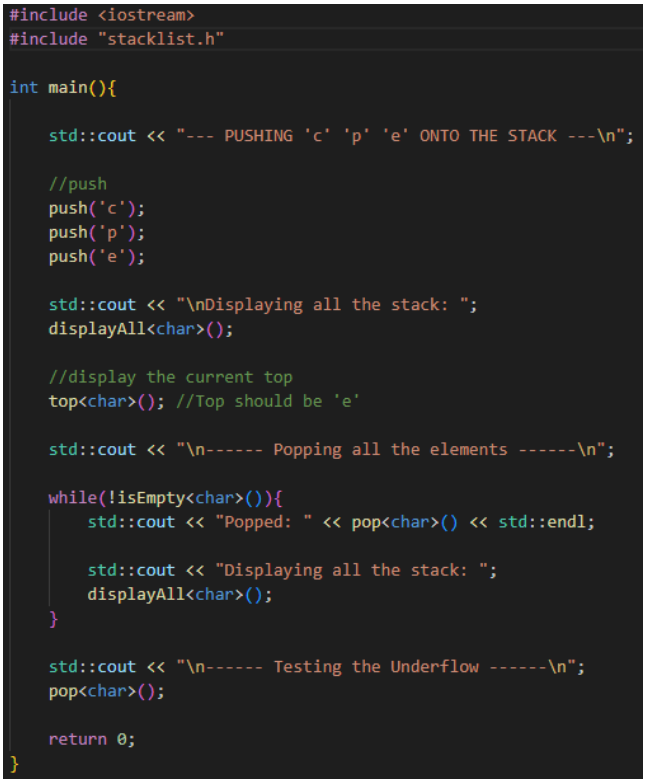
Stack Operations
1. PUSH
2. POP
3. TOP
4. isEMPTY
5. DISPLAY
Enter your choice: 5
Stack elements (top to bottom):
67

Stack Operations
1. PUSH
2. POP
3. TOP
4. isEMPTY
5. DISPLAY
Enter your choice: 1
Enter new value: 76

Stack Operations
1. PUSH
2. POP
3. TOP
4. isEMPTY
5. DISPLAY
Enter your choice: 5
Stack elements (top to bottom):
76 67
```

Table 4-5. Output of ILO B.1.

The output shows that the stack operations work correctly. The **user first pushes 67 into the stack**, making it the **top** element. Using the **TOP option displays 67**, while the **DISPLAY option shows the stack from top to bottom**. After **pushing another value, 76**, the **display updates to 76 67**, confirming that the most recently added element is placed on top, following the **Last In, First Out principle**.



```cpp
#include <iostream>
#include "stacklist.h"

int main(){

    std::cout << "--- PUSHING 'c' 'p' 'e' ONTO THE STACK ---\n";

    //push
    push('c');
    push('p');
    push('e');

    std::cout << "\nDisplaying all the stack: ";
    displayAll<char>();

    //display the current top
    top<char>(); //Top should be 'e'

    std::cout << "\n------ Popping all the elements ------\n";

    while(!isEmpty<char>()){
        std::cout << "Popped: " << pop<char>() << std::endl;

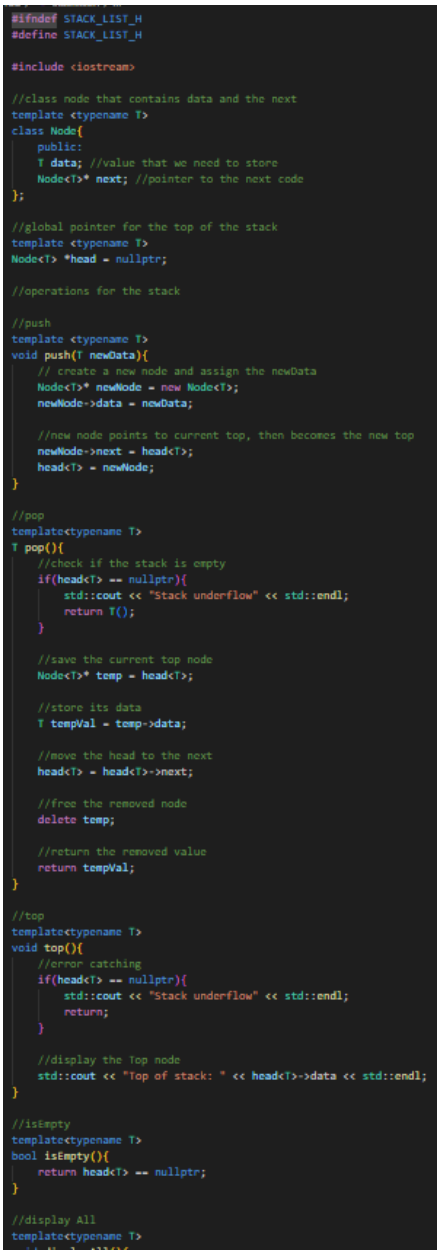
        std::cout << "Displaying all the stack: ";
        displayAll<char>();
    }

    std::cout << "\n------ Testing the Underflow ------\n";
    pop<char>();

    return 0;
}
```

Table 4-6. .cpp Code of ILO B.2.

The main.cpp file demonstrates how the stack operations work. It begins by **pushing the characters 'c', 'p', and 'e'** onto the stack. Because the stack follows the Last In, First Out (LIFO) principle, the last value inserted becomes the first value removed. **After pushing the elements, the program displays all the contents of the stack** and **then displays the current top element**. It then repeatedly removes each element using the **pop()** function inside a loop until the stack becomes empty. After each removal, the updated contents of the stack are displayed. Finally, **the program attempts to pop one more element from the empty stack** to demonstrate the underflow condition and display the corresponding error message.



```h
#ifndef STACK_LIST_H
#define STACK_LIST_H

#include <iostream>

//class node that contains data and the next
template <typename T>
class Node{
    public:
    T data; //value that we need to store
    Node<T>* next; //pointer to the next code
};

//global pointer for the top of the stack
template <typename T>
Node<T> *head = nullptr;

//operations for the stack

//push
template <typename T>
void push(T newData){
    // create a new node and assign the newData
    Node<T>* newNode = new Node<T>;
    newNode->data = newData;

    //new node points to current top, then becomes the new top
    newNode->next = head<T>;
    head<T> = newNode;
}

//pop
template<typename T>
T pop(){
    //check if the stack is empty
    if(head<T> == nullptr){
        std::cout << "Stack underflow" << std::endl;
        return T();
    }

    //save the current top node
    Node<T>* temp = head<T>;

    //store its data
    T tempVal = temp->data;

    //move the head to the next
    head<T> = head<T>->next;

    //free the removed node
    delete temp;

    //return the removed value
    return tempVal;
}

//top
template<typename T>
void top(){
    //error catching
    if(head<T> == nullptr){
        std::cout << "Stack underflow" << std::endl;
        return;
    }

    //display the Top node
    std::cout << "Top of stack: " << head<T>->data << std::endl;
}

//isEmpty
template<typename T>
bool isEmpty(){
    return head<T> == nullptr;
}

//display All
template<typename T>
```

Table 4-7. .h Code of ILO B.2.

The stacklist.h file contains the implementation of the stack using a linked list. It defines the **Node class**, where each node **stores a data value and a pointer to the next node**. A **global head pointer** is used to represent the top of the stack. The **push()** function inserts a new element by creating a node, storing the value, and placing it at the top of the stack. The **pop()** function removes the top element by saving the current top node, moving the head pointer to the next node, deleting the removed node, and returning its value. The **top()** function displays the value at the top of the stack without removing it, while the **isEmpty()** function checks whether the stack is empty by determining if the head pointer is nullptr.

```cpp
//display All
template<typename T>
void displayAll(){

    //display all nodes from top to bottom
    Node<T>* current = head<T>;

    while(current != nullptr){
        std::cout << current->data << " ";
        current = current->next;
    }

    std::cout << std::endl;
}
```

Table 4-8. .h file Modification of ILO B.2.

The main modification made to the program was the addition of the displayAll() function. The program only **displayed the top element of the stack using the top() function**. With this modification, the program now displays all the elements currently **stored in the stack after the push operation and after every pop operation.** The **displayAll()** function traverses the linked list from the head pointer to the last node, printing each value in order. This makes it easier to observe how the stack changes as elements are added and removed.

```
--- PUSHING 'c' 'p' 'e' ONTO THE STACK ---

Displaying all the stack: e p c
Top of stack: e

------ Popping all the elements ------
Popped: e
Displaying all the stack: p c
Popped: p
Displaying all the stack: c
Popped: c
Displaying all the stack:

------ Testing the Underflow ------
Stack underflow
```

Table 4-9. Output of ILO B.2.

The overall output begins by showing the elements that were pushed onto the stack, followed by the complete contents of the stack and the current top element. As each element is popped, the program displays the removed value and the updated stack contents. Once all elements have been removed, the program attempts one final pop() operation, resulting in a "Stack underflow" message, which confirms that the program correctly handles attempts to remove elements from an empty stack.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

```
//Array Stack
class ArrayStack {
private:
    char arr[100];
    int top;

public:
    //Start with an empty stack
    ArrayStack() {
        top = -1;
    }

    //Add symbol to the stack
    void push(char c) {
        arr[++top] = c;
    }

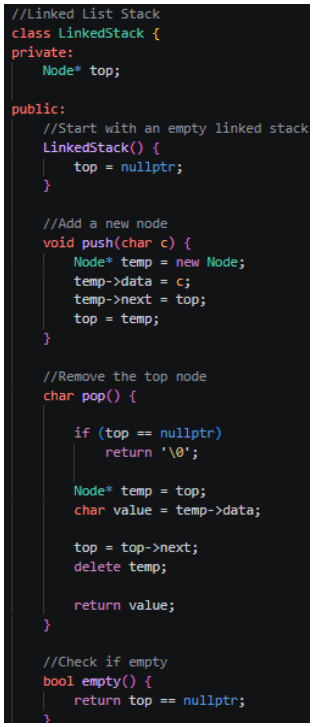
    //Remove the top symbol
    char pop() {
        if (top == -1)
            return '\0';

        return arr[top--];
    }

    //Check if the stack is empty
    bool empty() {
        return top == -1;
    }
```

Figure 4.1.Stack using Arrays

The array-based stack stores symbols in a fixed-size array and uses the `top` index to manage elements. Push and pop operations are simple and efficient, making it suitable for checking balanced symbols as long as the array does not exceed its maximum size.



```
//Linked List Stack
class LinkedStack {
private:
    Node* top;

public:
    //Start with an empty linked stack
    LinkedStack() {
        top = nullptr;
    }

    //Add a new node
    void push(char c) {
        Node* temp = new Node;
        temp->data = c;
        temp->next = top;
        top = temp;
    }

    //Remove the top node
    char pop() {

        if (top == nullptr)
            return '\0';

        Node* temp = top;
        char value = temp->data;

        top = top->next;
        delete temp;

        return value;
    }

    //Check if empty
    bool empty() {
        return top == nullptr;
    }
```

Figure 4.1.Stack using Linked Lists

The linked list stack stores symbols in dynamically allocated nodes connected by pointers. It performs push and pop operations by updating the top pointer, allowing the stack to grow or shrink as needed without being limited by a fixed array size.

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	Y	Symbol Balancing using Stack Enter an expression: (A+B)+(C-D) ----- Array Stack ----- Result: Balanced Symbols ----- Linked List Stack ----- Result: Balanced Symbols	All symbols are correctly matched and balanced.
((A+B)+(C-D)	N	Symbol Balancing using Stack Enter an expression: ((A+B)+(C-D) ----- Array Stack ----- Result: Not Balanced ----- Linked List Stack ----- Result: Not Balanced	One opening parenthesis is missing its pair.
((A+B)+[C-D])	Y	Symbol Balancing using Stack Enter an expression: ((A+B)+[C-D]) ----- Array Stack ----- Result: Balanced Symbols ----- Linked List Stack ----- Result: Balanced Symbols	Parentheses and brackets are correctly matched.

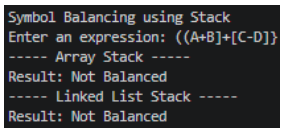
((A+B]+[C-D]}	N		The symbols are mismatched and not balanced.

Table 4-10. For supplementary output.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correct**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Used for grammar correction and replacing difference word usage**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.